计算机科学与技术学院

ARP 地址解析协议仿真软件

设计与实现

2024 _____

_____

_____

_____

2026 8

ARP IPv4 IP MAC Python
PySide6 ARP ARP ARP
Host A Host D

ARP “ ” “ ”
/
/ ARP

ARP PySide6

1 需求分析

1.1 课题背景与设计目标

ARP IPv4 MAC IP MAC ARP
IP ARP

“ ”

> “ARP ” Python 3 GUI PySide6 4 6
> IP

1.2 功能需求

FR-01		4 6 IP/MAC	
FR-02	ARP	REQUEST	
FR-03	ARP	IP REPLY	
FR-04	ARP	/ / 5 120	
FR-05			
FR-06			
FR-07		/ IPv4 MAC	
FR-08		4	

1-1

1.3 非功能需求与约束

- GUI
- Host A Host F host-1
-
- /
- IPv4 MAC
- 4 6

1.4 运行环境与操作流程

项目	配置
	macOS 26.6.2 Windows/Linux
	Python 3.14.3
GUI	PySide6 6.11.2
	threading.Thread queue.Queue threading.Timer
	pytest 9.1.1
	main.py “ ARP .command”

1-2

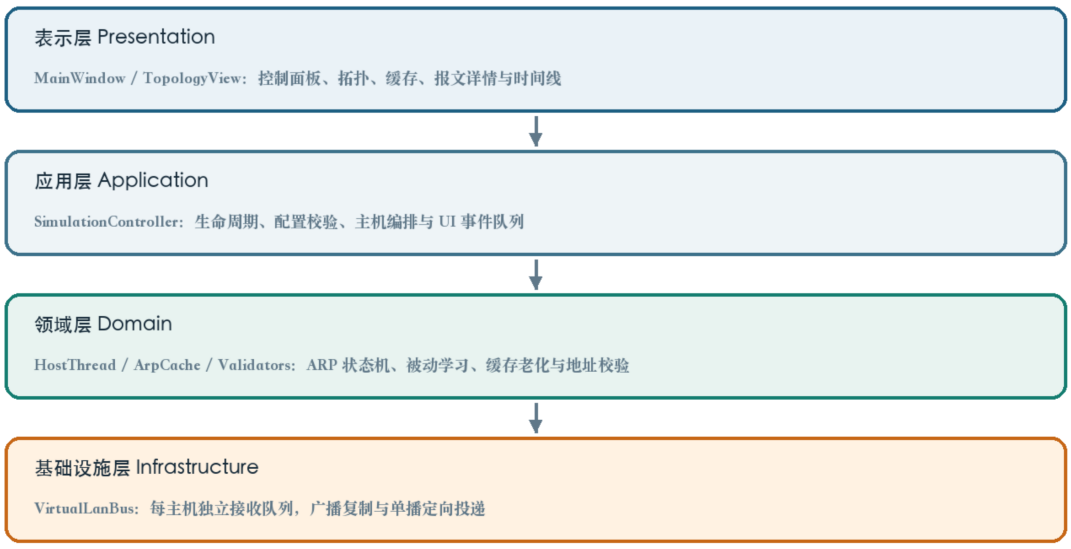
1. Host A Host D
2. “ ” IP “ ARP ”
3.
4.
5. IP

2 概要设计

2.1 总体架构

Qt ARP

GUI



2-1

2.2 模块划分与线程模型

models.py	HostConfig ArpPacket ArpCacheEntry SimulationEvent	
validators.py	IPv4 MAC	
cache.py		
lan_bus.py		
host_thread.py		
controller.py	/ /	
main_window.py		
topology_view.py	/	

2-1

HostConfig HostThread VirtualLanBus Queue[ArpPacket] ArpCache destination_host_id Qt

> GUI SimulationEvent

2.3 数据结构与状态设计

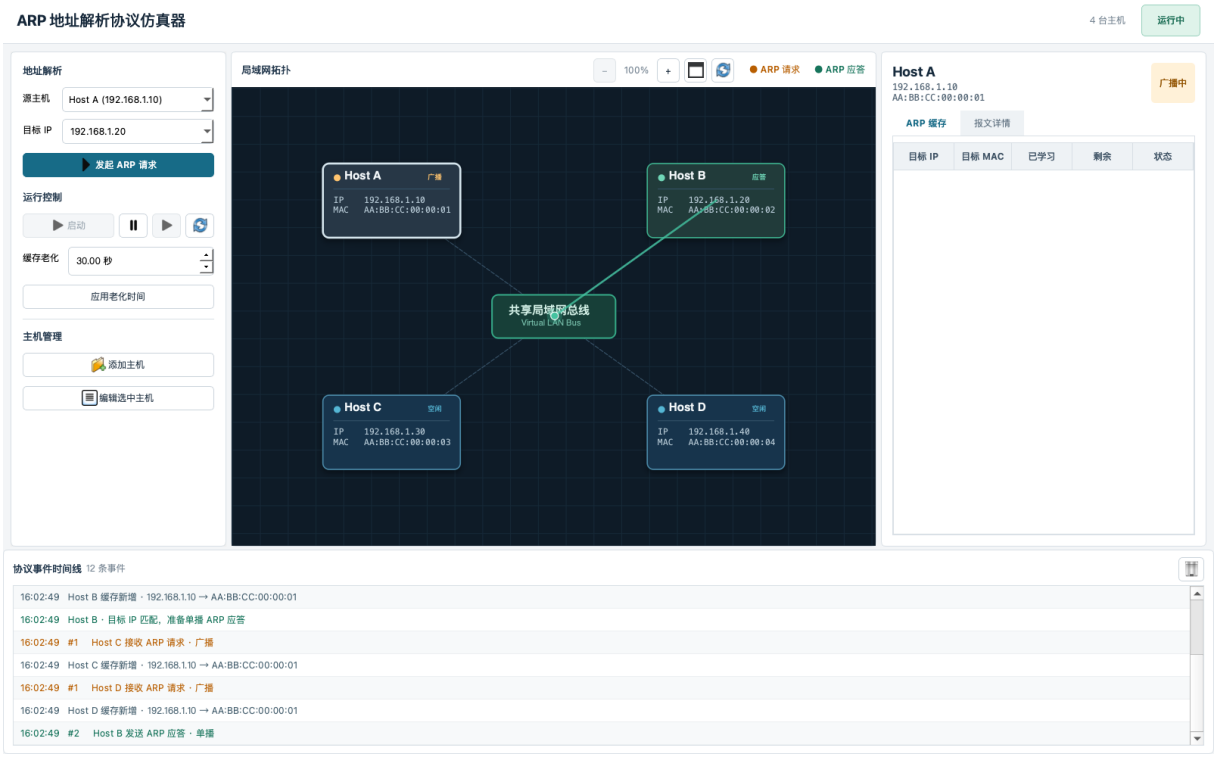
HostConfig	host_id name ip mac x y	
ArpPacket	packet_id opcode sender/target source/destination	ARP
ArpCacheEntry	ip mac learned_at last_seen state	IP-MAC
SimulationEvent	event_type timestamp host_id payload	GUI
dict[str, ArpCacheEntry]	IP	O(1)
dict[str, Queue]	host_id	

2-2

IDLE BROADCASTING REPLYING RESOLVED PAUSED TIMEOUT MISS NEW UPDATED HIT EXPIRED

2.4 图形界面设计

ARP



2-2 ARP /

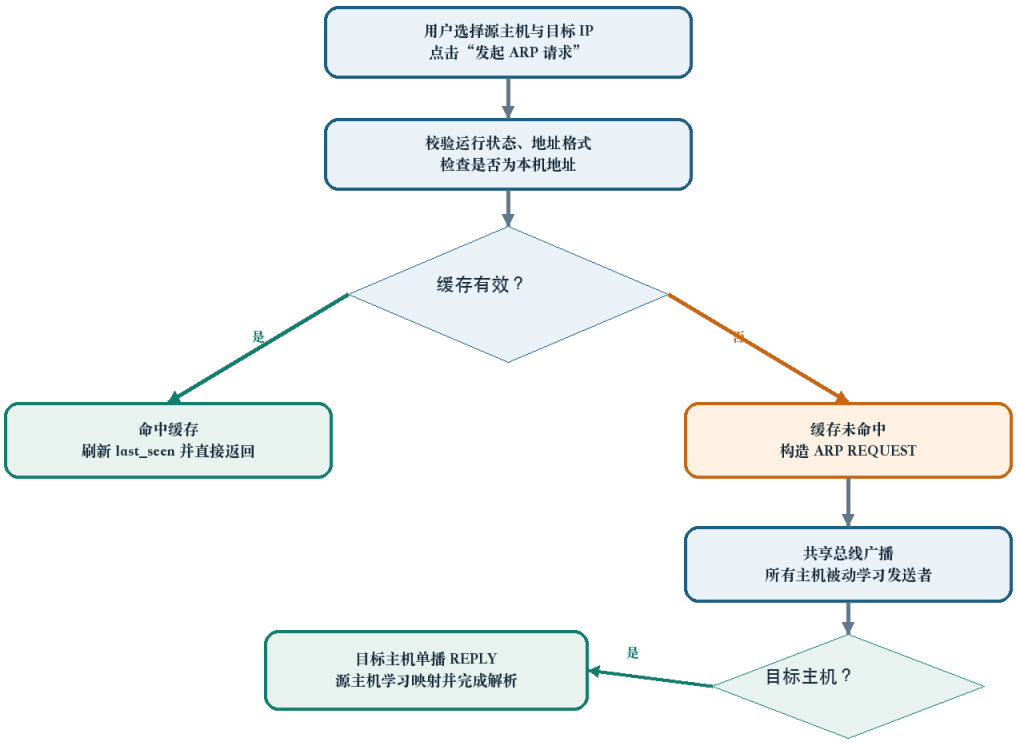
/ 100% 300%

3 □□□□□□□

3.1 ARP □□□□□□□

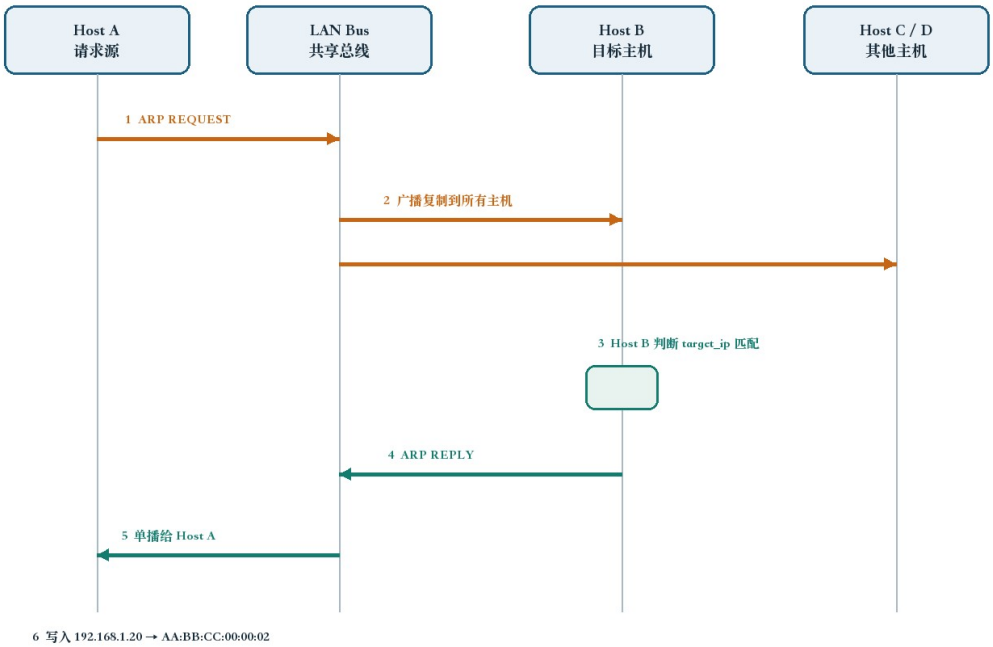
IPv4 SELF

last_seen HIT MAC REQUEST _pending



3-1 ARP

REQUEST sender_ip → sender_mac IP target_ip REPLY sender target destination_host_id RESOLVED request_timeout TIMEOUT



3-2 Host A Host B

	RP	RP
opcode	REQUEST	REPLY
sender_ip / sender_mac		
target_ip	IP	IP
target_mac	00:00:00:00:00:00	MAC
destination_host_id	None	host_id

3-1

3.2 ARP □□□□□□□□□

ArpCache RLock GUI learn() NEW UPDATED HIT lookup() last_seen expire() 200 ms

```
def learn(self, ip: str, mac: str, now: float | None = None) -> tuple[ArpCacheEntry, CacheChange]:
    now = self._clock() if now is None else now
    with self._lock:
        old = self._entries.get(ip)
        change = CacheChange.NEW if old is None else (
            CacheChange.UPDATED if old.mac != mac else CacheChange.HIT
        )
        entry = ArpCacheEntry(ip, mac, old.learned_at if old else now, now, change)
        self._entries[ip] = entry
        return entry, change

def lookup(self, ip: str, now: float | None = None) -> ArpCacheEntry | None:
    now = self._clock() if now is None else now
    with self._lock:
        entry = self._entries.get(ip)
        if entry is None or now - entry.last_seen >= self.aging_seconds:
            if entry is not None:
                del self._entries[ip]
            return None
        entry.last_seen = now
        entry.state = CacheChange.HIT
        return entry

def expire(self, now: float | None = None) -> list[ArpCacheEntry]:
    now = self._clock() if now is None else now
    expired: list[ArpCacheEntry] = []
    with self._lock:
        for ip, entry in list(self._entries.items()):
            if now - entry.last_seen >= self.aging_seconds:
                entry.state = CacheChange.EXPIRED
                expired.append(entry)
                del self._entries[ip]
    return expired
```

HostThread _paused_at paused_for learned_at last_seen started_at

```
def set_paused(self, paused: bool) -> None:
    if paused:
        if self._paused_at is None:
            self._paused_at = time.monotonic()
        self.pause_event.clear()
        self.status = "PAUSED"
    else:
        if self._paused_at is not None:
            paused_for = time.monotonic() - self._paused_at
```

```
            self.cache.shift_timestamps(paused_for)
            self._pending = {
                ip: (packet_id, started_at + paused_for)
                for ip, (packet_id, started_at) in self._pending.items()
            }
            self._paused_at = None
        self.pause_event.set()
        self.status = "IDLE"
    self._event(EventType.HOST_STATUS, {"status": self.status})
```

3.3 □□□□□□□/□□

VirtualLanBus host_id → Queue broadcast() unicast()
1.4 Timer close()

```
def broadcast(self, packet: ArpPacket) -> int:
    with self._lock:
        queues = list(self._queues.values())
    self._deliver_later(lambda: [q.put(packet) for q in queues])
    return len(queues)

def unicast(self, packet: ArpPacket) -> bool:
    with self._lock:
        q = self._queues.get(packet.destination_host_id or "")
    if q is None:
        return False
    self._deliver_later(lambda: q.put(packet))
    return True

def _deliver_later(self, delivery) -> None:
    if self.delivery_delay <= 0:
        delivery()
        return

    def run():
        try:
            with self._lock:
                if self._closed:
                    return
            delivery()
        finally:
            with self._lock:
                if timer in self._timers:
                    self._timers.remove(timer)

    timer = threading.Timer(self.delivery_delay, run)
    timer.daemon = True
    with self._lock:
        if self._closed:
            return
        self._timers.append(timer)
    timer.start()
```

> sender_ip/sender_mac

3.4 □□□□□□□□□□□

SimulationController GUI start() pause()/resume() stop()
join reset() stop()+start()

```
def start(self) -> None:
    with self._lock:
```

```
        if self.running:
            return
        self.bus = VirtualLanBus(self.delivery_delay)
        request_timeout = max(4.0, self.delivery_delay * 2 + 1.0)
        self.hosts = {
            host_id: HostThread(config, self.bus, self.events.put, self.aging_seconds, request_timeout)
            for host_id, config in self.configs.items()
        }
        for host in self.hosts.values():
            host.start()
        self.running, self.paused = True, False

    def pause(self) -> None:
        with self._lock:
            if not self.running:
                return
            self.paused = True
            for host in self.hosts.values():
                host.set_paused(True)

    def resume(self) -> None:
        with self._lock:
            if not self.running:
                return
            self.paused = False
            for host in self.hosts.values():
                host.set_paused(False)

    def stop(self) -> None:
        with self._lock:
            if self.bus:
                self.bus.close()
            for host in self.hosts.values():
                host.stop()
            for host in self.hosts.values():
                host.join(timeout=1.0)
            self.hosts.clear()
            self.bus = None
            self.running, self.paused = False, False

    def reset(self) -> None:
        self.stop()
        self.start()
```

events.put PACKET_SENT PACKET_RECEIVED CACHE_CHANGED HOST_STATUS ERROR drain host_id Host A

3.5 □□□□□□□□□□

TopologyView QGraphicsView/QGraphicsScene math.exp(delta × sensitivity) 8% 100% 300% “ ”

```
def _set_zoom(self, requested_zoom: float, anchor_pos) -> None:
    new_zoom = max(self.MIN_ZOOM, min(self.MAX_ZOOM, requested_zoom))
    if abs(new_zoom - self._zoom_factor) < 0.0001:
        return
    if new_zoom <= self.MIN_ZOOM:
        self.reset_view()
        return

    anchor_scene_before = self.mapToScene(anchor_pos)
    ratio = new_zoom / self._zoom_factor
    self.scale(ratio, ratio)
    self._zoom_factor = new_zoom
    anchor_scene_after = self.mapToScene(anchor_pos)
    center_after_scale = self.mapToScene(self.viewport().rect().center())
    anchor_correction = anchor_scene_before - anchor_scene_after
```

```
    self.centerOn(center_after_scale + anchor_correction)
    self._view_center = self.mapToScene(self.viewport().rect().center())
    self._update_pan_cursor()
    self.zoom_changed.emit(round(self._zoom_factor * 100))

def wheelEvent(self, event) -> None:
    delta = event.angleDelta().y()
    if not delta:
        delta = event.pixelDelta().y() * 4
    if delta:
        factor = math.exp(delta * self.WHEEL_SENSITIVITY)
        self._set_zoom(self._zoom_factor * factor, event.position().toPoint())
    event.accept()

def mousePressEvent(self, event) -> None:
    is_middle = event.button() == Qt.MouseButton.MiddleButton
    is_empty_left = (
        event.button() == Qt.MouseButton.LeftButton
        and self._interactive_item_at(event.position().toPoint()) is None
    )
    if self._zoom_factor > self.MIN_ZOOM and (is_middle or is_empty_left):
        self._pan_active = True
        self._pan_last_pos = event.position()
        self.viewport().setCursor(Qt.CursorShape.ClosedHandCursor)
        event.accept()
        return
    super().mousePressEvent(event)

def mouseMoveEvent(self, event) -> None:
    if self._pan_active and self._pan_last_pos is not None:
        movement = event.position() - self._pan_last_pos
        self._pan_last_pos = event.position()
        horizontal = self.horizontalScrollBar()
        vertical = self.verticalScrollBar()
        horizontal.setValue(horizontal.value() - round(movement.x()))
        vertical.setValue(vertical.value() - round(movement.y()))
        self._view_center = self.mapToScene(self.viewport().rect().center())
        event.accept()
        return
    super().mouseMoveEvent(event)
```

→ → REQUEST REPLY

start_provider/end_provider

```
def animate_packet(self, packet, mode: str, host_configs: dict[str, HostConfig]) -> None:
    source = self.nodes.get(packet.source_host_id)
    if not source or not self.hub:
        return
    if mode == "broadcast":
        targets = [node for host_id, node in self.nodes.items() if host_id != packet.source_host_id]
        color = QColor("#f4a340")
        activity = "REQUEST"
    else:
        target = self.nodes.get(packet.destination_host_id or "")
        targets = [target] if target else []
        color = QColor("#44c3a0")
        activity = "REPLY"

    self._active_packet_count += 1
    self.hub.set_activity(activity)

    def finish_packet():
        self._active_packet_count = max(0, self._active_packet_count - 1)
        if self._active_packet_count == 0 and self.hub:
            self.hub.set_activity("IDLE")

    def start_second_phase():
        if not targets:
            finish_packet()
            return
        remaining = {"count": len(targets)}
```

```
        def target_arrived():
            remaining["count"] -= 1
            if remaining["count"] == 0:
                finish_packet()

        for target_node in targets:
            self._animate_route(
                lambda: self.hub.center_point() if self.hub else QPointF(),
                lambda node=target_node: node.pos(),
                color,
                780,
                target_arrived,
            )

    self._animate_route(
        lambda node=source: node.pos(),
        lambda: self.hub.center_point() if self.hub else QPointF(),
        color,
        520,
        start_second_phase,
    )

def _animate_route(self, start_provider, end_provider, color: QColor, duration: int, on_finished) -> None:
    line = QGraphicsLineItem()
    line.setPen(QPen(color, 2.6))
    line.setOpacity(0.82)
    line.setZValue(1)
    self.scene.addItem(line)

    token = QGraphicsItemGroup()
    glow_color = QColor(color)
    glow_color.setAlpha(65)
    glow = QGraphicsEllipseItem(-10, -10, 20, 20, token)
    glow.setPen(QPen(Qt.PenStyle.NoPen))
    glow.setBrush(QBrush(glow_color))
    dot = QGraphicsEllipseItem(-5, -5, 10, 10, token)
    dot.setPen(QPen(QColor("#ffffff"), 1.1))
    dot.setBrush(QBrush(color))
    token.setZValue(2)
    self.scene.addItem(token)

    animation = QVariantAnimation(self)
    animation.setDuration(duration)
    animation.setStartValue(0.0)
    animation.setEndValue(1.0)
    animation.setEasingCurve(QEasingCurve.Type.InOutCubic)
    self._animations.append(animation)

    def update(progress):
        start = start_provider()
# ......                    ......
```

3.6 □□□□□□□

	/	
	1 24	
IPv4	ipaddress.IPv4Address	
MAC	- :	
	HostConfig IP/MAC	
	len(configs) < 6	
	target_ip _pending	PENDING
	close → bus.close → host.stop → join	Timer

3-2

4 调试分析

4.1 测试环境与测试方法

“ + GUI + ” pytest 0 GUI offscreen MainWindow Host A Host B

2026 8 27 .venv/bin/python -m pytest -q 4 passed in 0.22s Python GUI

4.2 测试数据及结果

	/		
T01	Host A ⟶ 192.168.1.20	REQUEST Host B REPLY Host A .20⟶...02	
T02	T01	CACHE_HIT	
T03	Host C	.10⟶...01	
T04	5	EXPIRED	
T05	3		
T06	Host A ⟶ 192.168.1.99	4	
T07	Host A ⟶ IP	SELF	
T08	IP/MAC		
T09	MAC AA-BB-CC-00-00-05		
T10	Host E Host F	5/6 7	
T11	200%		
T12	Host B/		

4-1

test_cache_lifecycle NEW HIT EXPIRED test_bus_broadcast_and_unicast test_arp_resolution_flow test_pause_excludes_aging_time A

4.3 算法复杂度分析

lookup	IP	O(1)	O(m)
learn	/	O(1)	O(m)
expire		O(m)	O(m)
	n	O(n)	O(n) /
		O(1)	O(1)
	6	O(n)	O(1)
		O(n)	O(n)

4-2 n≤6 m

O(1) O(m) m 200 ms

4.4 调试问题、解决方法与改进设想

GUI	SimulationEvent Queue	
	pending	
	request_timeout=max(4.0, 2×delivery_delay+1.0)	/
	provider	
host-1	HostConfig Host A F	

4-3

Wireshark ARP /

ARP Gratuitous ARP JSON/PCAP pytest-qt GUI Timer

5 课程设计总结

ARP “

MAC”

“ ” “ ” Qt SimulationEvent

“ ”

ARP

GUI

□□□□

[1] David C. Plummer. RFC 826: An Ethernet Address Resolution Protocol. IETF, 1982.

[2] R. Braden. RFC 1122: Requirements for Internet Hosts - Communication Layers. IETF, 1989.

[3] James F. Kurose, Keith W. Ross. Computer Networking: A Top-Down Approach. Pearson, 8th Edition, 2021.

[4] Python Software Foundation. Python 3 Documentation: threading, queue, dataclasses, ipaddress.

[5] The Qt Company. Qt for Python (PySide6) Documentation: Widgets and Graphics View Framework.

[6] . , 2026.

□□ A □□□□□

1876 Python

A.1 □□□□□□□models.py□

```
from __future__ import annotations

from dataclasses import dataclass, field
from enum import Enum
import time


class ArpOpcode(str, Enum):
    REQUEST = "REQUEST"
    REPLY = "REPLY"


class CacheChange(str, Enum):
    MISS = "MISS"
    NEW = "NEW"
    UPDATED = "UPDATED"
    HIT = "HIT"
    EXPIRED = "EXPIRED"


class EventType(str, Enum):
    PACKET_SENT = "PACKET_SENT"
    PACKET_RECEIVED = "PACKET_RECEIVED"
    CACHE_CHANGED = "CACHE_CHANGED"
    HOST_STATUS = "HOST_STATUS"
    ERROR = "ERROR"


@dataclass(frozen=True)
class HostConfig:
```

```
    host_id: str
    name: str
    ip: str
    mac: str
    x: float
    y: float


@dataclass(frozen=True)
class ArpPacket:
    packet_id: int
    opcode: ArpOpcode
    sender_ip: str
    sender_mac: str
    target_ip: str
    target_mac: str
    source_host_id: str
    destination_host_id: str | None = None
    created_at: float = field(default_factory=time.monotonic)


@dataclass
class ArpCacheEntry:
    ip: str
    mac: str
    learned_at: float
    last_seen: float
    state: CacheChange


@dataclass(frozen=True)
class SimulationEvent:
    event_type: EventType
    timestamp: float
    host_id: str
    payload: dict
```

A.2 □□□□□□□□□□host_thread.py□

```
class HostThread(threading.Thread):
    def __init__(self, config: HostConfig, bus: VirtualLanBus, emit: Callable[[SimulationEvent], None], aging_seconds: float = 30.0,
request_timeout: float = 4.0):
        super().__init__(name=f"host-{config.host_id}", daemon=True)
        self.config = config
        self.bus = bus
        self.emit = emit
        self.inbox = bus.register(config.host_id)
        self.cache = ArpCache(aging_seconds)
        self.request_timeout = request_timeout
        self.stop_event = threading.Event()
        self.pause_event = threading.Event()
        self.pause_event.set()
        self.status = "IDLE"
        self._last_expire_check = 0.0
        self._pending: dict[str, tuple[int, float]] = {}
        self._paused_at: float | None = None

    def _event(self, event_type: EventType, payload: dict) -> None:
        self.emit(SimulationEvent(event_type, time.time(), self.config.host_id, payload))

    def set_paused(self, paused: bool) -> None:
        if paused:
            if self._paused_at is None:
                self._paused_at = time.monotonic()
            self.pause_event.clear()
            self.status = "PAUSED"
        else:
            if self._paused_at is not None:
                paused_for = time.monotonic() - self._paused_at
                self.cache.shift_timestamps(paused_for)
                self._pending = {
                    ip: (packet_id, started_at + paused_for)
```

```
                for ip, (packet_id, started_at) in self._pending.items()
            }
            self._paused_at = None
        self.pause_event.set()
        self.status = "IDLE"
    self._event(EventType.HOST_STATUS, {"status": self.status})

def stop(self) -> None:
    self.stop_event.set()
    self.pause_event.set()

def request_resolution(self, target_ip: str) -> str:
    entry = self.cache.lookup(target_ip)
    if entry:
        self._event(EventType.CACHE_CHANGED, {"change": "HIT", "entry": entry})
        return "CACHE_HIT"
    if target_ip in self._pending:
        return "PENDING"
    self._event(EventType.CACHE_CHANGED, {"change": "MISS", "target_ip": target_ip})
    packet = ArpPacket(self.bus.next_packet_id(), ArpOpcode.REQUEST, self.config.ip, self.config.mac, target_ip, "00:00:00:00:00:00",
self.config.host_id)
    self._pending[target_ip] = (packet.packet_id, time.monotonic())
    self.status = "BROADCASTING"
    self._event(EventType.HOST_STATUS, {"status": self.status})
    count = self.bus.broadcast(packet)
    self._event(EventType.PACKET_SENT, {"packet": packet, "mode": "broadcast", "recipients": count})
    return "BROADCAST"

def run(self) -> None:
    try:
        while not self.stop_event.is_set():
            if not self.pause_event.wait(0.1):
                continue
            now = time.monotonic()
            if now - self._last_expire_check >= 0.2:
                for entry in self.cache.expire(now):
                    self._event(EventType.CACHE_CHANGED, {"change": "EXPIRED", "entry": entry})
                self._last_expire_check = now
            for target_ip, (packet_id, started_at) in list(self._pending.items()):
                if now - started_at >= self.request_timeout:
                    del self._pending[target_ip]
                    self.status = "TIMEOUT"
                    self._event(EventType.HOST_STATUS, {"status": self.status})
                    self._event(EventType.ERROR, {"message": f"ARP      #{packet_id}                    "})
                    self.status = "IDLE"
            try:
                packet = self.inbox.get(timeout=0.1)
            except queue.Empty:
                continue
            self._handle_packet(packet)
    finally:
        self.bus.unregister(self.config.host_id)

def _handle_packet(self, packet: ArpPacket) -> None:
    self._event(EventType.PACKET_RECEIVED, {"packet": packet, "mode": "broadcast" if packet.destination_host_id is None else "unicast"})
    if packet.sender_ip == self.config.ip and packet.sender_mac == self.config.mac:
        return
    entry, change = self.cache.learn(packet.sender_ip, packet.sender_mac)
    self._event(EventType.CACHE_CHANGED, {"change": change.value, "entry": entry})
    if packet.opcode is ArpOpcode.REQUEST and packet.target_ip == self.config.ip:
        reply = ArpPacket(self.bus.next_packet_id(), ArpOpcode.REPLY, self.config.ip, self.config.mac, packet.sender_ip, packet.sender_mac,
self.config.host_id, packet.source_host_id)
        self.status = "REPLYING"
        self._event(EventType.HOST_STATUS, {"status": self.status})
        self.bus.unicast(reply)
        self._event(EventType.PACKET_SENT, {"packet": reply, "mode": "unicast", "recipients": 1})
        self.status = "IDLE"
    elif packet.opcode is ArpOpcode.REPLY and packet.destination_host_id == self.config.host_id:
        self._pending.pop(packet.sender_ip, None)
        self.status = "RESOLVED"
        self._event(EventType.HOST_STATUS, {"status": self.status})
        self.status = "IDLE"
```

A.3 □□□□□□□□□□controller.py□

```
class SimulationController:
    def __init__(self, event_queue: queue.Queue[SimulationEvent] | None = None, delivery_delay: float = 1.4):
        self.events = event_queue or queue.Queue()
        self.configs: dict[str, HostConfig] = {h.host_id: h for h in DEFAULT_HOSTS}
        self.bus: VirtualLanBus | None = None
        self.hosts: dict[str, HostThread] = {}
        self.aging_seconds = 30.0
        self.delivery_delay = delivery_delay
        self.running = False
        self.paused = False
        self._lock = threading.RLock()

    def start(self) -> None:
        with self._lock:
            if self.running:
                return
            self.bus = VirtualLanBus(self.delivery_delay)
            request_timeout = max(4.0, self.delivery_delay * 2 + 1.0)
            self.hosts = {
                host_id: HostThread(config, self.bus, self.events.put, self.aging_seconds, request_timeout)
                for host_id, config in self.configs.items()
            }
            for host in self.hosts.values():
                host.start()
            self.running, self.paused = True, False

    def pause(self) -> None:
        with self._lock:
            if not self.running:
                return
            self.paused = True
            for host in self.hosts.values():
                host.set_paused(True)

    def resume(self) -> None:
        with self._lock:
            if not self.running:
                return
            self.paused = False
            for host in self.hosts.values():
                host.set_paused(False)

    def stop(self) -> None:
        with self._lock:
            if self.bus:
                self.bus.close()
            for host in self.hosts.values():
                host.stop()
            for host in self.hosts.values():
                host.join(timeout=1.0)
            self.hosts.clear()
            self.bus = None
            self.running, self.paused = False, False

    def reset(self) -> None:
        self.stop()
        self.start()

    def set_aging_seconds(self, seconds: float) -> None:
        if not 5 <= seconds <= 120:
            raise ValueError("               5  120       ")
        self.aging_seconds = seconds
        for host in self.hosts.values():
            host.cache.aging_seconds = seconds

    def resolve(self, host_id: str, target_ip: str) -> str:
        if not self.running:
            raise RuntimeError("            ")
        if self.paused:
            raise RuntimeError("                    ")
        host = self.hosts.get(host_id)
        if host is None:
```

```
            raise ValueError("            ")
        target_ip = normalize_ip(target_ip)
        if target_ip == host.config.ip:
            return "SELF"
        return host.request_resolution(target_ip)

    def add_host(self, name: str, ip: str, mac: str) -> HostConfig:
        if len(self.configs) >= 6:
            raise ValueError("        6       ")
        name, ip, mac = validate_host(name, ip, mac)
        if any(h.ip == ip for h in self.configs.values()):
            raise ValueError("IP             ")
        if any(h.mac == mac for h in self.configs.values()):
            raise ValueError("MAC              ")
        index = len(self.configs) + 1
        extra_positions = {5: (105, 295), 6: (655, 295)}
        x, y = extra_positions[index]
        config = HostConfig(f"host-{index}", name, ip, mac, x, y)
        self.configs[config.host_id] = config
        if self.running:
            self.reset()
        return config

    def update_host(self, host_id: str, name: str, ip: str, mac: str) -> HostConfig:
        if host_id not in self.configs:
            raise ValueError("          ")
        name, ip, mac = validate_host(name, ip, mac)
        for other_id, other in self.configs.items():
            if other_id != host_id and other.ip == ip:
                raise ValueError("IP             ")
            if other_id != host_id and other.mac == mac:
                raise ValueError("MAC              ")
        old = self.configs[host_id]
        config = HostConfig(host_id, name, ip, mac, old.x, old.y)
        self.configs[host_id] = config
        was_running = self.running
        if was_running:
            self.reset()
        return config

    def cache_snapshot(self, host_id: str):
        host = self.hosts.get(host_id)
        return host.cache.snapshot() if host else []

    def close(self) -> None:
        self.stop()
```

A.4 □□□□□□test_core.py□

```
import queue
import time

from arp_simulator.cache import ArpCache
from arp_simulator.controller import SimulationController
from arp_simulator.lan_bus import VirtualLanBus
from arp_simulator.models import ArpOpcode, ArpPacket, CacheChange


def test_cache_lifecycle():
    now = [0.0]
    cache = ArpCache(aging_seconds=5, clock=lambda: now[0])
    _, change = cache.learn("192.168.1.2", "AA:BB:CC:00:00:02")
    assert change is CacheChange.NEW
    assert cache.lookup("192.168.1.2") is not None
    now[0] = 6
    assert cache.expire() and cache.lookup("192.168.1.2") is None


def test_bus_broadcast_and_unicast():
    bus = VirtualLanBus(delivery_delay=0)
    qa, qb = bus.register("a"), bus.register("b")
    packet = ArpPacket(1, ArpOpcode.REQUEST, "192.168.1.1", "AA:BB:CC:00:00:01", "192.168.1.2", "00:00:00:00:00:00", "a")
```

```
    assert bus.broadcast(packet) == 2
    assert qa.get_nowait() == packet and qb.get_nowait() == packet
    reply = ArpPacket(2, ArpOpcode.REPLY, "192.168.1.2", "AA:BB:CC:00:00:02", "192.168.1.1", "AA:BB:CC:00:00:01", "b", "a")
    assert bus.unicast(reply)
    assert qa.get_nowait() == reply
    assert qb.empty()


def test_arp_resolution_flow():
    controller = SimulationController(delivery_delay=0)
    controller.start()
    try:
        result = controller.resolve("host-1", "192.168.1.20")
        assert result == "BROADCAST"
        deadline = time.monotonic() + 2
        events = []
        while time.monotonic() < deadline:
            try:
                events.append(controller.events.get(timeout=0.05))
            except queue.Empty:
                pass
            if any(e.event_type.value == "CACHE_CHANGED" and e.host_id == "host-1" and e.payload.get("change") == "NEW" for e in events):
                break
        assert any(e.event_type.value == "PACKET_SENT" and e.payload["packet"].opcode is ArpOpcode.REPLY for e in events)
        assert controller.cache_snapshot("host-1")
        assert controller.resolve("host-1", "192.168.1.20") == "CACHE_HIT"
    finally:
        controller.close()


def test_pause_excludes_aging_time():
    controller = SimulationController(delivery_delay=0)
    controller.set_aging_seconds(5)
    controller.start()
    try:
        controller.hosts["host-1"].cache.learn("192.168.1.99", "AA:BB:CC:00:00:99")
        controller.pause()
        host = controller.hosts["host-1"]
        host._paused_at -= 10
        controller.resume()
        assert controller.cache_snapshot("host-1")
    finally:
        controller.close()
```

□□□□□□□

- source .venv/bin/activate && python main.py “ ARP .command”
- → Host A Host B → / → Host C → → 5 /
  → 192.168.1.99 → / Host E
- .md
- main.py arp_simulator/ tests/